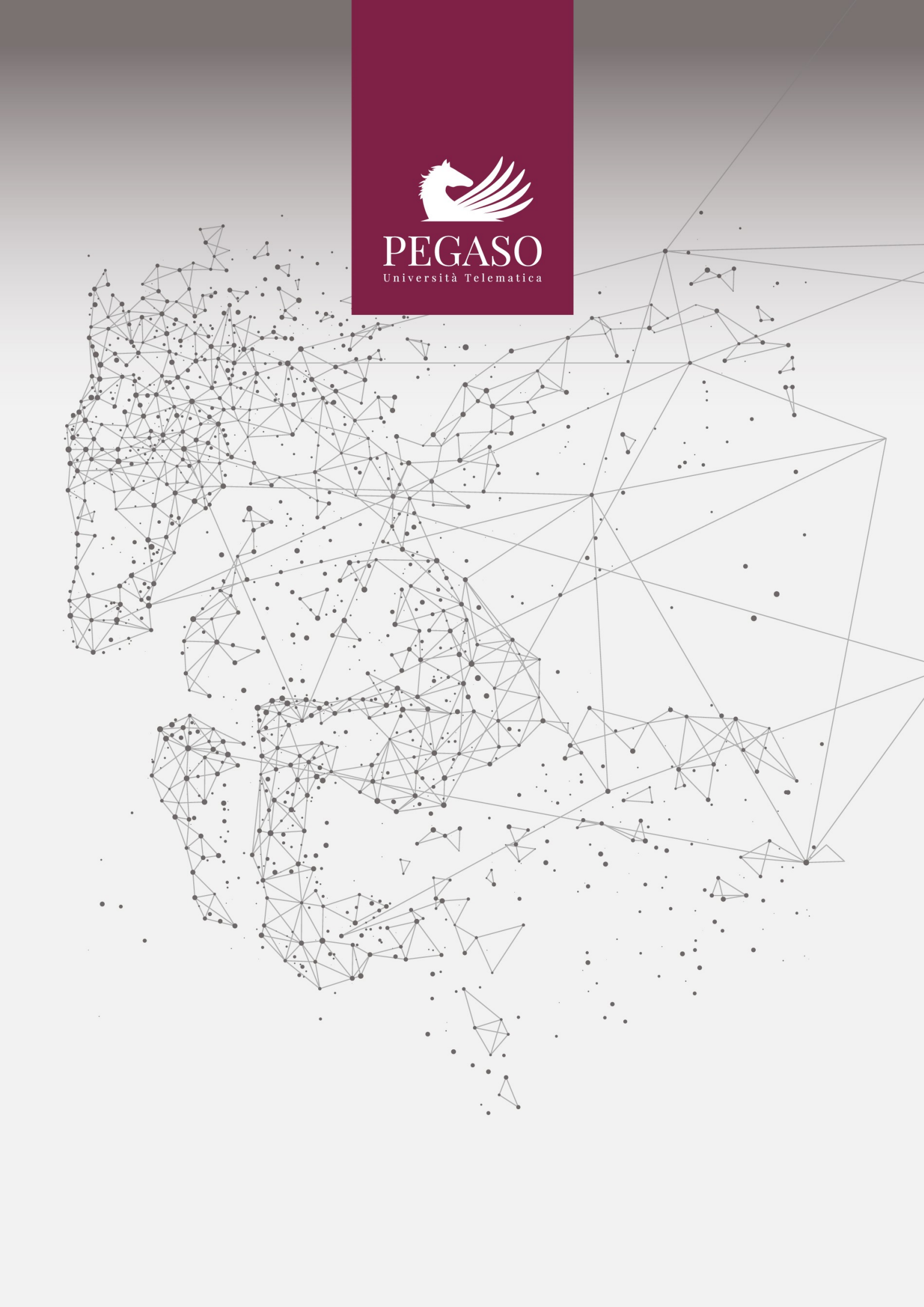

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. TODO LIST 3
2. TODO LIST CON SPOSTAMENTO 8
3. FETCH API IN REACT NATIVE 12
4. TIC TAC TOE 14
5. NAVIGATION BAR 18
BIBLIOGRAFIA 27
SITOGRAFIA 28

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. TODO LIST

Di seguito il codice per realizzare una ToDo List in React Native:

```
import React, { useState } from 'react';
import { StyleSheet, Text, TextInput, TouchableOpacity, View } from 'react-native';

export default function App() {
  const [task, setTask] = useState('');
  const [taskList, setTaskList] = useState([]);

  function handleAddTask() {
    setTaskList([...taskList, task]);
    setTask('');
  }

  function handleRemoveTask(indexToRemove) {
    setTaskList(taskList.filter((_, index) => index !== indexToRemove));
  }

  return (
    <View style={styles.container}>
      <Text style={styles.title}>To-Do List</Text>
      <View style={styles.inputContainer}>
        <TextInput
          style={styles.input}
          placeholder="Add a new task"
          value={task}
          onChangeText={text => setTask(text)}
        />
        <TouchableOpacity style={styles.addButton} onPress={handleAddTask}>
          <Text style={styles.addButtonText}>Add</Text>
        </TouchableOpacity>
      </View>
      <View style={styles.taskListContainer}>
        {taskList.map((task, index) => (
          <View style={styles.taskListItem} key={index}>
            <Text style={styles.taskListItemText}>{task}</Text>
            <TouchableOpacity
              style={styles.removeButton}
              onPress={() => handleRemoveTask(index)}
            >
              <Text style={styles.removeButtonText}>Remove</Text>
            </TouchableOpacity>
          </View>
        ))}
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'flex-start',
    paddingTop: 50,
  },
  title: {
    fontSize: 24,
    fontWeight: 'bold',
    marginBottom: 20,
  },
  inputContainer: {
    flexDirection: 'row',
    alignItems: 'center',
    justifyContent: 'space-between',
    marginBottom: 20,
    width: '80%',
  },
  input: {
    borderColor: 'gray',
    borderWidth: 1,
    padding: 10,
    width: '70%',
  },
  addButton: {
    backgroundColor: 'blue',
    borderRadius: 5,
    padding: 10,
    width: '25%',
    alignItems: 'center',
  },
  addButtonText: {
    color: 'white',
    fontWeight: 'bold',
  },
  taskListContainer: {
    width: '80%',
  },
  taskListItem: {
    flexDirection: 'row',
    alignItems: 'center',
    justifyContent: 'space-between',
    borderColor: 'gray',
    borderWidth: 1,
    padding: 10,
    marginBottom: 10,
  },
  taskListItemText: {
    flex: 1,
    fontSize: 16,
  },
  removeButton: {
    backgroundColor: 'red',
    borderRadius: 5,
    padding: 10,
    marginLeft: 10,
  },
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
  removeButtonText: {
    color: 'white',
    fontWeight: 'bold',
  },
});
```

Ecco la renderizzazione:

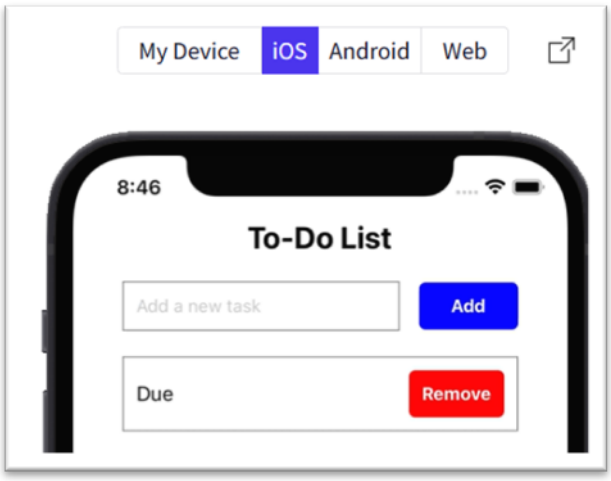


Figura 1 *- ToDo List*

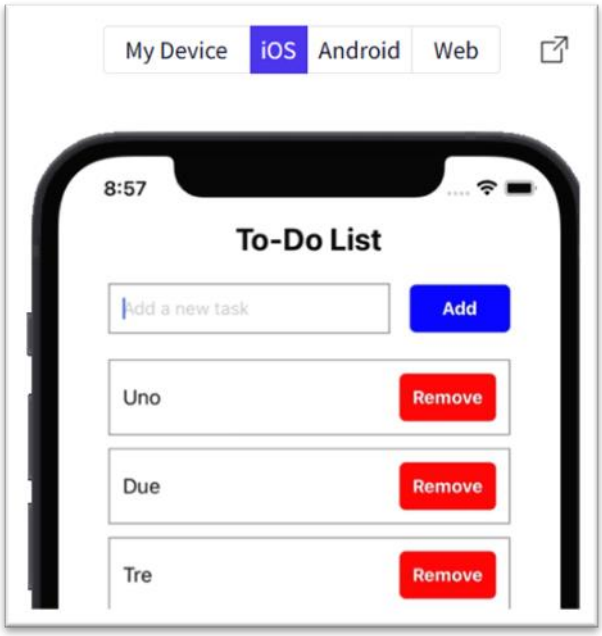


Figura 2 *- ToDo List*

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

In realtà sarebbe preferibile utilizzare dei files separati per meglio organizzare il codice, in particolare:

App.js

In questo file, abbiamo definito il componente principale dell'applicazione App, che visualizza un titolo e il componente TaskList. Il componente TaskList è definito in un file separato e viene importato all'interno di App. Il componente App restituisce un elemento View che contiene il titolo e il componente TaskList.

TaskList.js

In questo file, abbiamo definito il componente TaskList che viene utilizzato per visualizzare l'elenco delle attività nella lista. Il componente TaskList restituisce un elemento View che contiene un campo di testo e un pulsante di aggiunta (TouchableOpacity) per aggiungere una nuova attività. Inoltre, il componente TaskList contiene anche un elemento View che visualizza l'elenco delle attività nella lista, utilizzando il componente Task che è definito in un file separato.

Il componente TaskList utilizza l'hook useState per gestire lo stato dell'applicazione, ovvero l'elenco delle attività nella lista. Quando l'utente aggiunge una nuova attività, la funzione handleAddTask aggiunge l'attività alla lista utilizzando il valore corrente del campo di testo e aggiorna lo stato dell'applicazione con il nuovo elenco delle attività; quando l'utente rimuove un'attività dalla lista, la funzione handleRemoveTask rimuove l'attività dall'elenco utilizzando l'indice dell'attività e aggiorna lo stato dell'applicazione con il nuovo elenco delle attività.

Inoltre, il file TaskList.js importa il componente Task definito in un file separato e lo utilizza per visualizzare ciascuna attività nella lista.

Task.js:

Il componente Task è dotato di un pulsante di rimozione che viene attivato quando l'utente fa clic su di esso. Quando l'utente fa clic sul pulsante di rimozione, viene chiamata la funzione onRemove passata come argomento, che rimuove l'attività dalla lista.

Ovviamente, è anche possibile suddividere il codice di ciascun componente in due file separati, uno per il codice **JSX** e l'altro per lo stile **CSS**: questo può essere particolarmente utile per mantenere il codice dell'applicazione organizzato e facile da gestire, soprattutto per applicazioni più grandi.

Ad esempio, si potrebbe avere un file Task.js:

```
import React from 'react';
import { StyleSheet, Text, TouchableOpacity, View } from 'react-native';
import styles from './TaskStyles';
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
export default function Task({ task, onRemove }) {
  return (
    <View style={styles.taskListItem}>
      <Text style={styles.taskListItemText}>{task}</Text>
      <TouchableOpacity style={styles.removeButton} onPress={onRemove}>
        <Text style={styles.removeButtonText}>Remove</Text>
      </TouchableOpacity>
    </View>
  );
}
```

Il file TaskStyles.js invece è del tipo:

```
import { StyleSheet } from 'react-native';

const styles = StyleSheet.create({
  taskListItem: {
    flexDirection: 'row',
    alignItems: 'center',
    justifyContent: 'space-between',
    borderColor: 'gray',
    borderWidth: 1,
    padding: 10,
    marginBottom: 10,
  },
  taskListItemText: {
    flex: 1,
    fontSize: 16,
  },
  removeButton: {
    backgroundColor: 'red',
    borderRadius: 5,
    padding: 10,
    marginLeft: 10,
  },
  removeButtonText: {
    color: 'white',
    fontWeight: 'bold',
  },
});

export default styles;
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. TODO LIST con spostamento

Di seguito il codice per realizzare una ToDo List in React Native con lo spostamento delle attività in alto ed in basso (omettiamo la parte di style, analoga alla precedente):

```
export default function App() {
  const [task, setTask] = useState('');
  const [taskList, setTaskList] = useState([]);

  function handleAddTask() {
    setTaskList([...taskList, task]);
    setTask('');
  }

  function handleRemoveTask(indexToRemove) {
    setTaskList(taskList.filter((_, index) => index !== indexToRemove));
  }

  function handleMoveTask(currentIndex, newIndex) {
    const newTaskList = [...taskList];
    const [removedTask] = newTaskList.splice(currentIndex, 1);
    newTaskList.splice(newIndex, 0, removedTask);
    setTaskList(newTaskList);
  }

  return (
    <View style={styles.container}>
      <Text style={styles.title}>To-Do List</Text>
      <View style={styles.inputContainer}>
        <TextInput
          style={styles.input}
          placeholder="Add a new task"
          value={task}
          onChangeText={text => setTask(text)}
        />
        <TouchableOpacity style={styles.addButton} onPress={handleAddTask}>
          <Text style={styles.addButtonText}>Add</Text>
        </TouchableOpacity>
      </View>
      <View style={styles.taskListContainer}>
        {taskList.map((task, index) => (
          <View style={styles.taskListItem} key={index}>
            <Text style={styles.taskListItemText}>{task}</Text>
            <View style={styles.taskListButtons}>
              <TouchableOpacity
                style={[styles.moveButton, { marginRight: 5 }]}
                disabled={index === 0}
                onPress={() => handleMoveTask(index, index - 1)}
              >
                <Text style={styles.moveButtonText}>Up</Text>
              </TouchableOpacity>
              <TouchableOpacity
                style={[styles.moveButton, { marginLeft: 5 }]}
```

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
                disabled={index === taskList.length - 1}
                onPress={() => handleMoveTask(index, index + 1)}
              >
                <Text style={styles.moveButtonText}>Down</Text>
              </TouchableOpacity>
              <TouchableOpacity
                style={styles.removeButton}
                onPress={() => handleRemoveTask(index)}
              >
                <Text style={styles.removeButtonText}>Remove</Text>
              </TouchableOpacity>
            </View>
          </View>
        ))}
      </View>
    </View>
  );
}
```

È stata aggiunta la funzione handleMoveTask, che accetta l'indice del task da spostare (currentIndex) e l'indice della nuova posizione del task (newIndex) come input. All'interno di questa funzione, una copia dell'array taskList viene creata e il task corrente viene rimosso dall'array utilizzando il metodo splice. Successivamente, il task rimosso viene inserito nella nuova posizione utilizzando il metodo splice e infine il nuovo array taskList viene impostato utilizzando setTaskList.

Gli stili taskListButtons, moveButton per i pulsanti di movimento "Su" e "Giu" e moveButtonText per il testo all'interno dei pulsanti di movimento devono essere aggiunti, così come lo stile del pulsante di rimozione removeButton deve essere modificato per includere il testo "Remove".

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Ecco la renderizzazione:

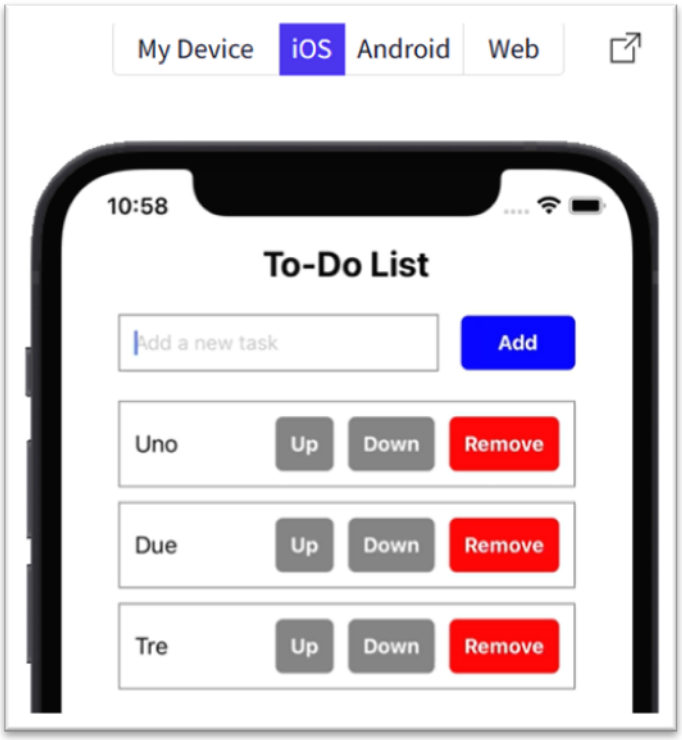


Figura 3 *- ToDo List*

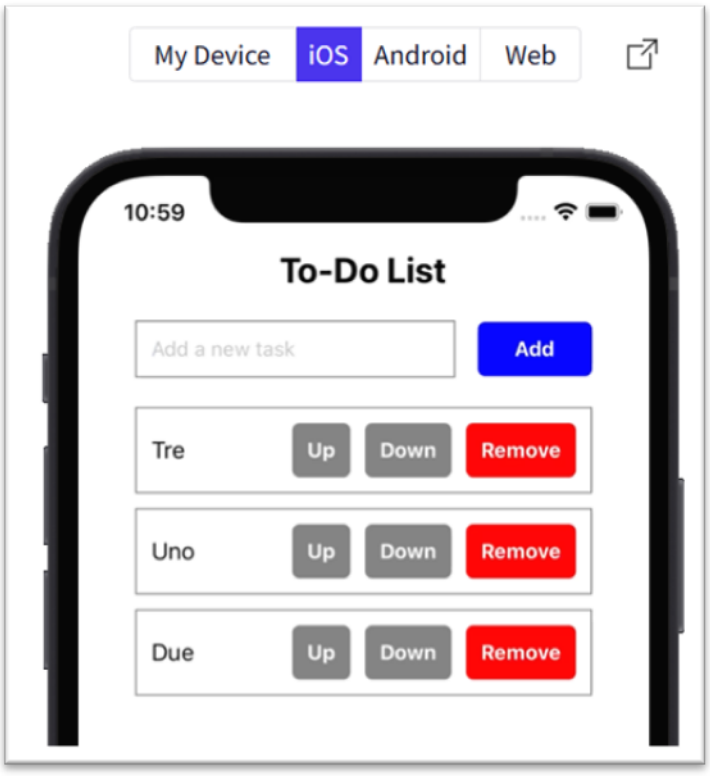


Figura 4 *- ToDo List*

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

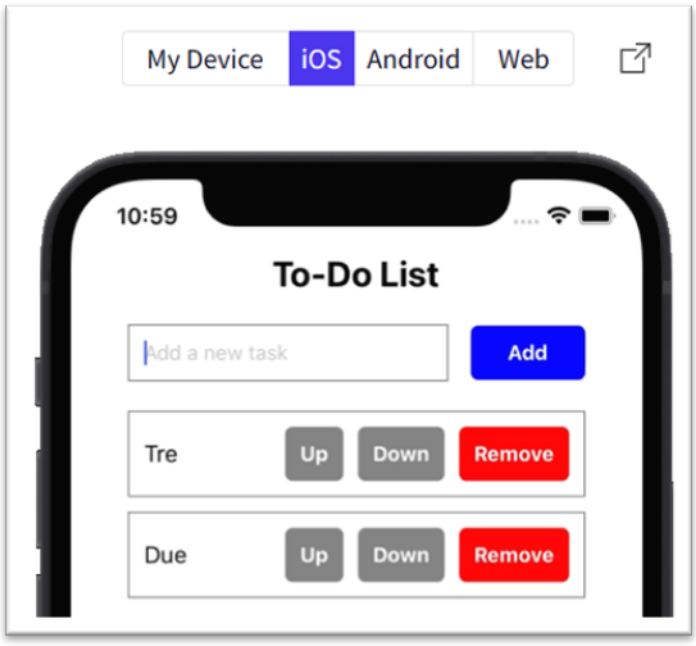


Figura 5 *- ToDo List*

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Fetch API in React Native

Il seguente codice rappresenta un componente React Native che recupera un'immagine casuale dall'API esterna "picsum.photos" e la mostra all'interno di un'interfaccia utente.

```
import React, { useState, useEffect } from 'react';
import { StyleSheet, Image, View, ActivityIndicator } from 'react-native';

export default function App() {
  const [imageData, setImageData] = useState(null);
  const [isLoading, setIsLoading] = useState(true);

  useEffect(() => {
  fetch('https://picsum.photos/200')
    .then((response) => {
      setImageData(response.url);
      setIsLoading(false);
    })
    .catch((error) => {
      console.log(error);
    });
}, [setImageData, setIsLoading]);


  return (
    <View style={styles.container}>
      {isLoading ? (
        <ActivityIndicator size="large" color="#0000ff" />
      ) : (
        <Image style={styles.image} source={{uri:imageData?imageData:''}} />
      )}
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
  image: {
    width: 200,
    height: 200,
  },
});
```

Il codice utilizza gli hook di React useState e useEffect per gestire gli stati della componente e recuperare l'immagine da un'API esterna. In particolare, l'hook useEffect viene utilizzato per effettuare una richiesta HTTP utilizzando fetch() e aggiornare lo stato della componente quando i dati vengono recuperati.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

La componente utilizza inoltre l'indicatore di caricamento ActivityIndicator di React Native per mostrare un feedback visivo quando l'immagine non è ancora stata caricata.

La componente utilizza la funzione useState per definire gli stati della componente come "imageData" e "isLoading". L'effetto collaterale useEffect viene chiamato quando l'immagine deve essere recuperata. In particolare, viene utilizzato fetch() per recuperare i dati dall'API esterna. Una volta che i dati sono stati recuperati con successo, la funzione setImageData viene utilizzata per aggiornare lo stato "imageData" con l'URL dell'immagine recuperata e la funzione setIsLoading viene utilizzata per impostare lo stato "isLoading" su "false". Se c'è un errore durante la richiesta, viene mostrato un messaggio di errore tramite console.log().

Infine, la funzione di rendering restituisce un componente View che contiene l'indicatore di caricamento ActivityIndicator quando l'immagine non è ancora stata caricata e l'immagine stessa quando i dati sono stati recuperati con successo. L'immagine viene mostrata utilizzando la componente Image di React Native e viene passato il parametro source con l'URL dell'immagine recuperata.

Ecco come viene renderizzata l'immagine:



Figura 6 *- React Native*

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. TIC TAC TOE

Di seguito il codice del Tic Tac Toe:

```
import React, { useState } from 'react';
import { StyleSheet, Text, View, TouchableOpacity } from 'react-native';

export default function App() {
  const [board, setBoard] = useState(Array(9).fill(null));
  const [currentPlayer, setCurrentPlayer] = useState('X');
  const [winner, setWinner] = useState(null);

  const handleSquareClick = (index) => {
    const newBoard = [...board];
    if (winner || newBoard[index]) return;
    newBoard[index] = currentPlayer;
    setBoard(newBoard);
    const newWinner = calculateWinner(newBoard);
    if (newWinner) {
      setWinner(newWinner);
    } else {
      setCurrentPlayer(currentPlayer === 'X' ? 'O' : 'X');
    }
  };

  const handleResetClick = () => {
    setBoard(Array(9).fill(null));
    setCurrentPlayer('X');
    setWinner(null);
  };

  const renderSquare = (index) => (
    <TouchableOpacity style={styles.square}
       onPress={() => handleSquareClick(index)}>
      <Text style={styles.squareText}>{board[index]}</Text>
    </TouchableOpacity>
  );

  const calculateWinner = (squares) => {
    const lines = [
      [0, 1, 2],
      [3, 4, 5],
      [6, 7, 8],
      [0, 3, 6],
      [1, 4, 7],
      [2, 5, 8],
      [0, 4, 8],
      [2, 4, 6],
    ];
    for (let i = 0; i < lines.length; i++) {
      const [a, b, c] = lines[i];
      if (squares[a] && squares[a] ===
       squares[b] && squares[a] === squares[c]) {
         return squares[a];
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
      }
    }
    return null;
  };

  return (
    <View style={styles.container}>
      <View style={styles.board}>
        <View style={styles.row}>
          {renderSquare(0)}
          {renderSquare(1)}
          {renderSquare(2)}
        </View>
        <View style={styles.row}>
          {renderSquare(3)}
          {renderSquare(4)}
          {renderSquare(5)}
        </View>
        <View style={styles.row}>
          {renderSquare(6)}
          {renderSquare(7)}
          {renderSquare(8)}
        </View>
      </View>
      {winner ? (
        <View style={styles.result}>
          <Text style={styles.resultText}>{winner} ha vinto!</Text>
          <TouchableOpacity style={styles.resetButton}
             onPress={handleResetClick}>
            <Text style={styles.resetButtonText}>Gioca di nuovo</Text>
          </TouchableOpacity>
        </View>
      ) : (
        <View style={styles.turn}>
          <Text style={styles.turnText}>{currentPlayer}'s turno</Text>
        </View>
      )}
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
  board: {
    width: 300,
    height: 300,
    borderWidth: 1,
    borderColor: 'black',
    flexDirection: 'column',
  },
  row: {
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
    flex: 1,
    flexDirection: 'row',
  },
  square: {
  flex: 1,
   borderWidth: 1,
   borderColor: 'black',
   alignItems: 'center',
   justifyContent: 'center',
  },
  squareText: {
   fontSize: 48,
  },
  turn: {
   marginTop: 50,
  },
  turnText: {
   fontSize: 24,
  },
  result: {
   marginTop: 50,
   alignItems: 'center',
  },
  resultText: {
   fontSize: 24,
  },
  resetButton: {
   backgroundColor: 'blue',
   padding: 10,
   borderRadius: 5,
   marginTop: 20,
  },
  resetButtonText: {
   color: 'white',
   fontSize: 16,
  },
});
```

L'applicazione è strutturata in 2 componenti riutilizzabili, Square e Board, che migliorano la leggibilità e la manutenibilità del codice.

L'applicazione tiene traccia dello stato del gioco attraverso una matrice 3x3 di quadrati, rappresentata dalla variabile board. Inoltre, è presente una variabile currentPlayer che indica il giocatore corrente, che inizia con "X".

Quando un giocatore clicca su un quadrato vuoto, l'applicazione cambia lo stato del gioco e passa al giocatore successivo. L'applicazione controlla anche se un giocatore ha vinto il gioco e mostra un messaggio di vittoria quando necessario. Inoltre, è presente un pulsante "Gioca di nuovo" che consente ai giocatori di iniziare una nuova partita.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

L'applicazione utilizza gli hooks di React, useState() e useEffect(), per creare e gestire lo stato locale dell'applicazione. In particolare, useState() viene utilizzato per creare lo stato locale per la matrice board e la variabile currentPlayer. useEffect() viene utilizzato per eseguire effetti collaterali in risposta a cambiamenti di stato, come la verifica del vincitore dopo ogni mossa.

Di seguito la renderizzazione:

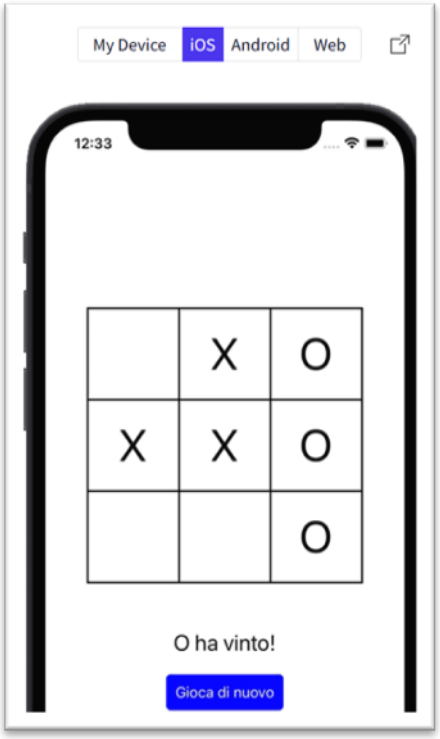


Figura 7 *- Tic Tac Toe*

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

5. Navigation bar

La navigation bar è uno dei componenti più importanti per la navigazione all'interno di un'app mobile e React Native offre diverse librerie per la creazione di questo tipo di componenti. Alcune delle librerie più popolari per la navigazione in React Native sono:

- **React Navigation**: questa libreria offre una vasta gamma di opzioni di navigazione, tra cui stack, tab e drawer. La sua API è molto flessibile e personalizzabile, il che la rende adatta per la maggior parte dei progetti. È compatibile con entrambe le piattaforme, iOS e Android, ed è facile da usare grazie alla documentazione dettagliata. Inoltre, è possibile integrare librerie esterne come React Native Gesture Handler e React Native Reanimated per migliorare la fluidità delle animazioni.
- **React Native Router Flux**: questa libreria offre una navigazione basata su scene, che rappresentano uno stato della vista dell'applicazione. Supporta la navigazione basata su stack e scena, ed è facile da usare grazie a un'API intuitiva. La libreria offre anche funzionalità avanzate come la gestione dello stato globale e la sincronizzazione del percorso URL con la navigazione.
- **Native Navigation**: questa libreria è stata sviluppata da Airbnb ed è focalizzata sulla velocità e le prestazioni. Supporta solo la navigazione basata su stack, ma offre un'esperienza di navigazione molto fluida e veloce grazie al rendering nativo dei componenti. La libreria ha un'API semplice ed è facile da usare, ma ha meno opzioni di personalizzazione rispetto alle altre librerie.

In generale, React Navigation è la scelta più popolare e versatile, mentre React Native Router Flux e Native Navigation sono opzioni valide per progetti più specifici. La scelta dipende dalle esigenze specifiche del progetto e dalle preferenze del team di sviluppo.

Analizziamo React Navigation. Abbiamo 3 opzioni di navigazione: stack, tab e drawer.

- **Stack Navigator**: lo stack navigator gestisce la navigazione basata su stack, che è una pila di schermate sovrapposte. Ogni volta che si naviga verso una nuova schermata, questa viene aggiunta alla pila. Quando si preme il pulsante "Indietro", la schermata corrente viene rimossa dalla pila e si ritorna alla schermata precedente. Questo tipo di navigazione è ideale per flussi di lavoro lineari, in cui l'utente segue un percorso ben definito.
- **Tab Navigator**: Il tab navigator gestisce la navigazione basata su tab, in cui ogni tab rappresenta una schermata separata. Gli utenti possono passare da una scheda all'altra toccando la scheda

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

corrispondente. Questo tipo di navigazione è utile per le app che hanno più funzionalità o viste diverse che devono essere facilmente accessibili.

- **Drawer Navigator**: Il drawer navigator gestisce la navigazione basata su drawer, che è un menu laterale che si apre da un lato dello schermo. L'utente può aprire il menu laterale toccando un pulsante di menu nella navigation bar. Il drawer può contenere qualsiasi tipo di contenuto, come voci di menu, opzioni di configurazione e altro ancora. Questo tipo di navigazione è utile per le app che hanno molte funzionalità e opzioni di navigazione.

In generale, la scelta tra stack, tab e drawer dipende dalle esigenze specifiche dell'applicazione. Se l'applicazione ha un flusso di lavoro lineare, lo stack navigator potrebbe essere la scelta migliore. Se l'applicazione ha molte funzionalità diverse, il tab navigator potrebbe essere la scelta migliore. Se l'applicazione ha molte opzioni di navigazione, il drawer navigator potrebbe essere la scelta migliore. Tuttavia, è anche possibile combinare diversi tipi di navigatori per creare un'esperienza di navigazione più complessa e personalizzata.

Stack

Ecco il codice:

```
import * as React from 'react';
import { View, Text,Button } from 'react-native';
import { NavigationContainer } from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native-stack';

function HomeScreen({ navigation }) {
  return (
    <View style={{ flex: 1, alignItems: 'center', justifyContent: 'center' }}>
      <Text>Home Screen</Text>
      <Button
        title="Go to Details"
        onPress={() => navigation.navigate('Details')}
      />
      <Button
        title="Go to Settings"
        onPress={() => navigation.navigate('Settings')}
      />
    </View>
  );
}

function DetailsScreen() {
  return (
    <View style={{ flex: 1, alignItems: 'center', justifyContent: 'center' }}>
      <Text>Details Screen</Text>
    </View>
  );
}

function SettingsScreen() {
  return (
    <View style={{ flex: 1, alignItems: 'center', justifyContent: 'center' }}>
      <Text>Settings Screen</Text>
    </View>
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
  );
}

const Stack = createNativeStackNavigator();

function App() {
  return (
    <NavigationContainer>
      <Stack.Navigator>
        <Stack.Screen name="Home" component={HomeScreen} />
        <Stack.Screen name="Details" component={DetailsScreen} />
        <Stack.Screen name="Settings" component={SettingsScreen} />
      </Stack.Navigator>
    </NavigationContainer>
  );
}

export default App;
```

Questo codice utilizza React Navigation per creare un'applicazione React Native con una schermata di avvio, una schermata dei dettagli e una schermata delle impostazioni. La navigazione tra le schermate è gestita da uno stack navigator, che consente all'utente di andare avanti e indietro tra le schermate come se fossero una pila di carte.

In particolare, la schermata di avvio (HomeScreen) mostra il testo "Home Screen" e due pulsanti: uno per andare alla schermata dei dettagli e uno per andare alla schermata delle impostazioni. La schermata dei dettagli (DetailsScreen) mostra il testo "Details Screen" e la schermata delle impostazioni (SettingsScreen) mostra il testo "Settings Screen".

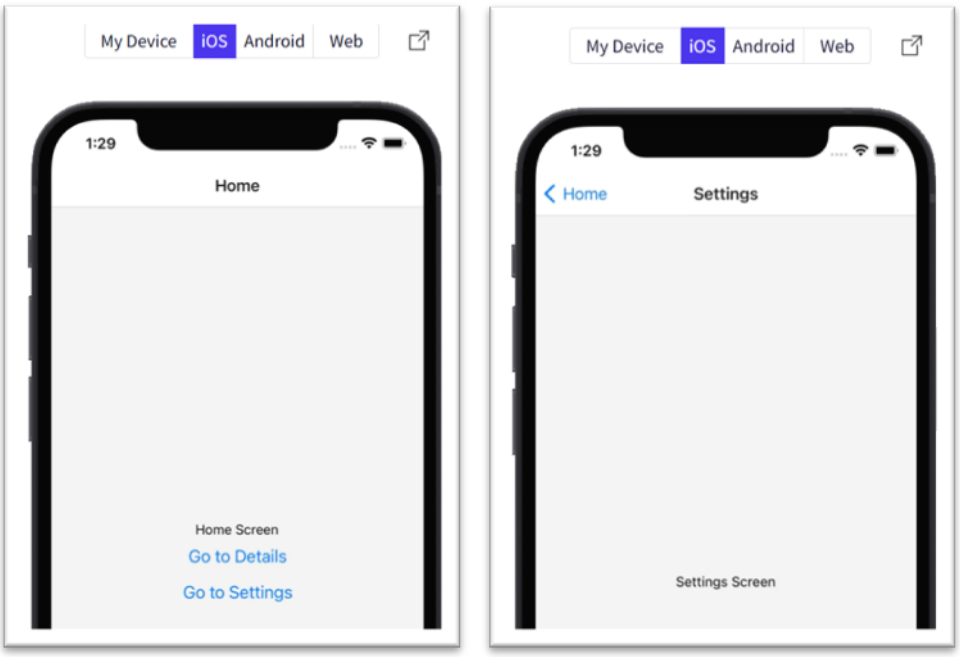
Il componente NavigationContainer di React Navigation fornisce il contesto di navigazione per l'applicazione, mentre il componente createNativeStackNavigator viene utilizzato per creare lo stack navigator.

All'interno dello stack navigator, abbiamo tre oggetti Screen che rappresentano le schermate: Home, Details e Settings. Ciascuno di questi oggetti Screen definisce il nome della schermata e la componente React Native da visualizzare quando la schermata viene selezionata. Nella componente HomeScreen, i due pulsanti utilizzano il metodo navigation.navigate per navigare verso le rispettive schermate.

Infine, abbiamo la componente App, che rende il componente NavigationContainer contenente lo stack navigator e lo esporta come componente predefinito.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).



Le dependencies che occorre inserire sono:

```
{
  "dependencies": {
    "@expo/vector-icons": "^13.0.0",
    "@react-native-community/masked-view": "*",
    "react-native-gesture-handler": "~2.8.0",
    "react-native-pager-view": "6.0.1",
    "react-native-paper": "^4.7.2",
    "react-native-reanimated": "~2.12.0",
    "react-native-safe-area-context": "4.4.1",
    "react-native-screens": "~3.18.0",
    "react-native-tab-view": "^3.0.0",
    "@react-navigation/bottom-tabs": "6.3.1",
    "@react-navigation/drawer": "6.4.1",
    "@react-navigation/elements": "1.3.3",
    "@react-navigation/material-bottom-tabs": "6.2.1",
    "@react-navigation/material-top-tabs": "6.2.1",
    "@react-navigation/native-stack": "6.6.1",
    "@react-navigation/native": "6.0.10",
    "@react-navigation/stack": "6.2.1"
  }
}
```

Material Bottom Tabs

Ecco di seguito il codice:

```
import * as React from 'react';
import { View, Text, Button, SafeAreaView, StyleSheet } from 'react-native';
import { NavigationContainer } from '@react-navigation/native';
import { createBottomTabNavigator } from '@react-navigation/bottom-tabs';

function HomeScreen() {
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
  return (
    <View style={{ flex: 1, alignItems: 'center', justifyContent: 'center' }}>
      <Text>Home Screen</Text>
    </View>
  );
}

function DetailsScreen() {
  return (
    <View style={{ flex: 1, alignItems: 'center', justifyContent: 'center' }}>
      <Text>Details Screen</Text>
    </View>
  );
}

function SettingsScreen() {
  return (
    <View style={{ flex: 1, alignItems: 'center', justifyContent: 'center' }}>
      <Text>Settings Screen</Text>
    </View>
  );
}

const Tab = createBottomTabNavigator();

function App() {
  return (
    <SafeAreaView style={styles.container}>
      <NavigationContainer>
        <Tab.Navigator>
          <Tab.Screen name="Home" component={HomeScreen} />
          <Tab.Screen name="Details" component={DetailsScreen} />
          <Tab.Screen name="Settings" component={SettingsScreen} />
        </Tab.Navigator>
      </NavigationContainer>
    </SafeAreaView>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    paddingBottom: 20
  }
});

export default App;
```

Questo codice utilizza React Navigation per creare un'applicazione React Native con una navigazione a tab nella parte inferiore dell'applicazione. La navigazione a tab consente all'utente di passare da una schermata all'altra toccando le schede della barra di navigazione.

In particolare, abbiamo creato tre schermate (HomeScreen, DetailsScreen e SettingsScreen) come nel precedente esempio di navigazione a stack.

PEGASO
Università Telematica

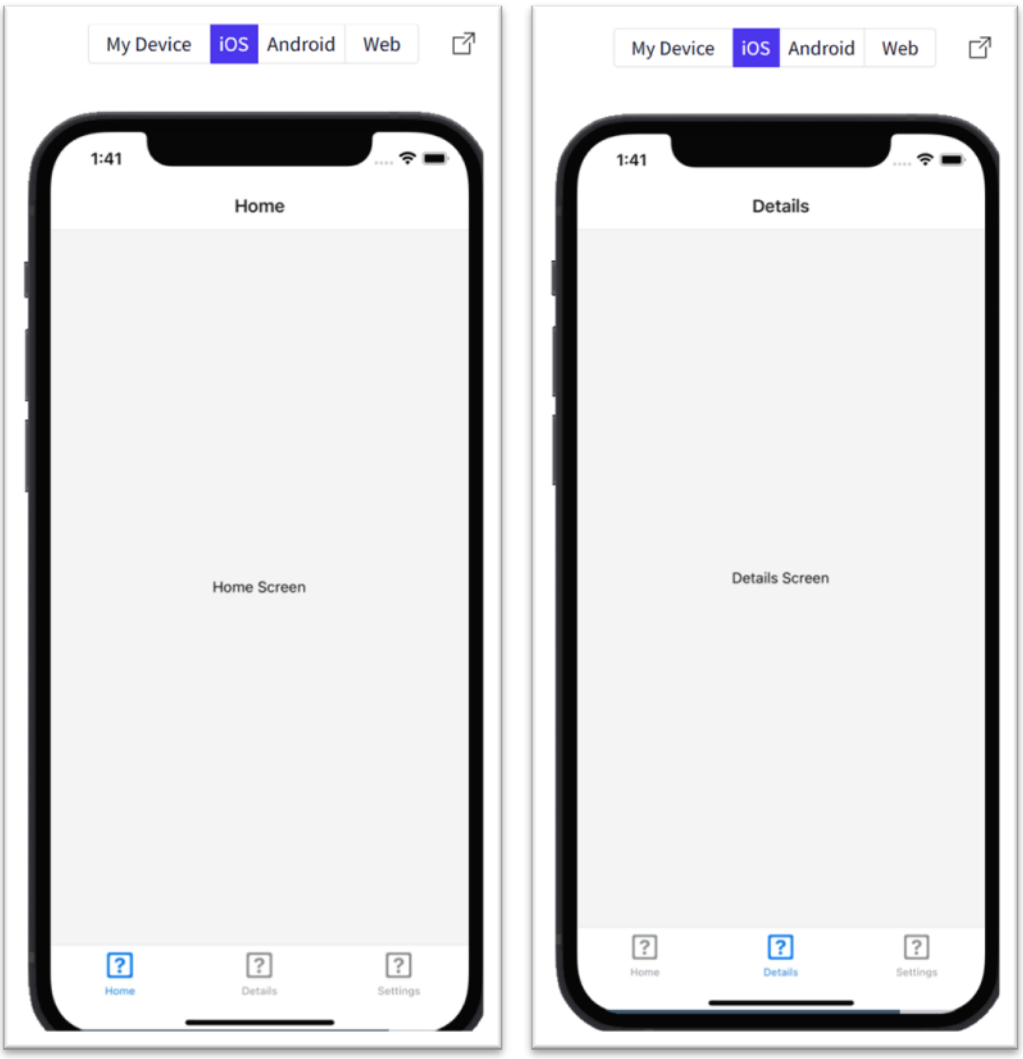
Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Abbiamo anche importato il componente createBottomTabNavigator di React Navigation e lo abbiamo utilizzato per creare un nuovo oggetto Tab che rappresenta la navigazione a tab.

Successivamente, abbiamo utilizzato SafeAreaView per avvolgere il contenitore delle schede di navigazione a tab e impostato uno stile CSS (styles.container) per definire la proprietà paddingBottom. Questa proprietà è stata impostata su 20, il che garantisce che la barra di navigazione a tab sia posizionata correttamente nella parte inferiore dell'applicazione.

Infine, abbiamo utilizzato Tab.Navigator per creare un contenitore per le schermate di navigazione a tab e Tab.Screen per definire ciascuna schermata della navigazione a tab e assegnarle un nome.

In questo esempio, le tre schermate sono visualizzate come schede nella parte inferiore dell'applicazione e l'utente può passare da una schermata all'altra toccando la scheda corrispondente.



Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Drawer

Di seguito il codice per un drawer navigator:

```
import * as React from 'react';
import { View, Text } from 'react-native';
import { NavigationContainer } from '@react-navigation/native';
import {
  createDrawerNavigator,
  DrawerContentScrollView,
  DrawerItemList,
  DrawerItem,
} from '@react-navigation/drawer';

function Feed() {
  return (
    <View style={{ flex: 1, justifyContent: 'center', alignItems: 'center' }}>
      <Text>Feed Screen</Text>
    </View>
  );
}

function Article() {
  return (
    <View style={{ flex: 1, justifyContent: 'center', alignItems: 'center' }}>
      <Text>Article Screen</Text>
    </View>
  );
}

function CustomDrawerContent(props) {
  return (
    <DrawerContentScrollView {...props}>
      <DrawerItemList {...props} />
      <DrawerItem label="Help" onPress={() => alert('Link to help')} />
    </DrawerContentScrollView>
  );
}

const Drawer = createDrawerNavigator();

function MyDrawer() {
  return (
    <Drawer.Navigator
      useLegacyImplementation
      drawerContent={(props) => <CustomDrawerContent {...props} />}
    >
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
      <Drawer.Screen name="Feed" component={Feed} />
      <Drawer.Screen name="Article" component={Article} />
    </Drawer.Navigator>
  );
}

export default function App() {
  return (
    <NavigationContainer>
      <MyDrawer />
    </NavigationContainer>
  );
}
```

Questo codice utilizza React Navigation per creare un'applicazione React Native con una navigazione a drawer. La navigazione a drawer consente all'utente di aprire e chiudere un menu laterale a comparsa per accedere alle varie sezioni dell'applicazione.

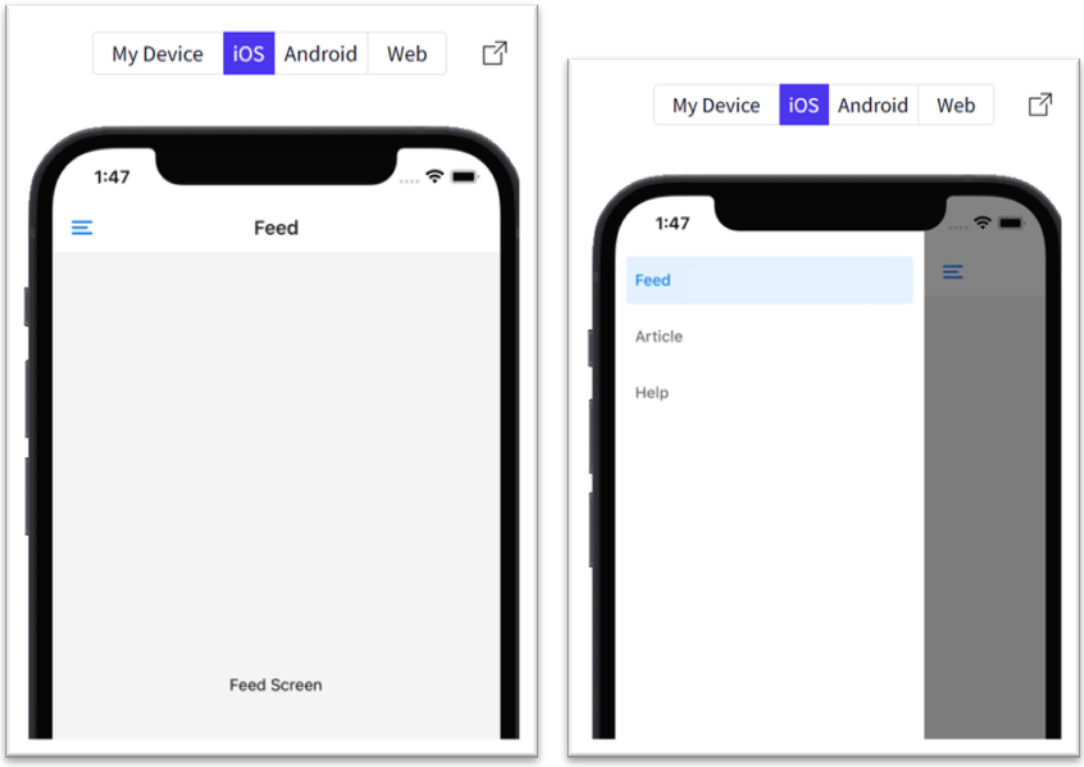
In particolare, abbiamo creato due schermate (Feed e Article) come nel precedente esempio di navigazione a stack. Abbiamo anche importato il componente createDrawerNavigator di React Navigation e lo abbiamo utilizzato per creare un nuovo oggetto Drawer che rappresenta la navigazione a drawer.

Successivamente, abbiamo definito una funzione CustomDrawerContent che serve a creare il contenuto del drawer personalizzato. Questa funzione accetta le proprietà props e restituisce un DrawerContentScrollView che rappresenta il contenitore dello ScrollView del drawer e un DrawerItemList che rappresenta la lista degli elementi del drawer. Abbiamo aggiunto anche un DrawerItem per il collegamento alla sezione di aiuto dell'applicazione.

Infine, abbiamo utilizzato Drawer.Navigator per creare un contenitore per le schermate di navigazione a drawer e Drawer.Screen per definire ciascuna schermata della navigazione a drawer e assegnarle un nome. In questo esempio, le due schermate sono visualizzate come elementi del drawer laterale, che si apre e si chiude facendo clic sull'icona a tre linee nella parte superiore sinistra dell'applicazione.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).



Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- V. Roberto, M. Frailis, A. Gugliotta, P. Omero. Introduzione alle Tecnologie Web, McGraw-Hill, 2010.
- A. S. Tanembaum. Computer Networks (4 ed.). Prentice Hall (4 ed.), 2003.
- Marco Andreini, Patrizia Andronico, Arianna Del Soldato, Cristian Lucchesi, Maurizio Martinelli, Giuseppe Vasarelli. Manuale corso di Tecnologie Web Istituto di Informatica e Telematica del CNR, 2005.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Sitografia

- HTML 4.01 Specification: http://www.w3.org/TR/html401/
- HTML5: http://www.w3.org/TR/html5/
- Javascript: JavaScript Tutorial (w3schools.com)
- HTML: http://www.w3schools.com/html/default.asp
- CSS: https://www.w3schools.com/css/default.asp
- CSS: http://www.w3.org/TR/CSS/
- NODEJS: https://nodejs.org
- REACT: https://it.legacy.reactjs.org/
- REACT NATIVE: https://reactnative.dev/

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).